

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Динамические списки»

Выполнили студенты группы 25BBB2

Лампетов А.С.

Ковтун А.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Динамические списки

Цель работы:

изучение динамических списков (односвязный список, стек, очередь, приоритетная очередь) и реализация основных операций над ними на языке Go.

Лабораторное задание:

1. Реализовать приоритетную очередь путём добавления элемента в список в соответствии с приоритетом объекта (объект с большим приоритетом располагается перед объектом с меньшим приоритетом).
2. На основе приведённого кода реализовать структуру данных «Очередь» (FIFO).
3. На основе приведённого кода реализовать структуру данных «Стек» (LIFO).

Ход работы:

В качестве базового элемента всех реализованных структур используется односвязный узел Node, содержащий информационное поле Inf, поле приоритета Prior (используется только приоритетной очередью) и указатель Next на следующий элемент списка. На основе этого узла реализованы три структуры данных: приоритетная очередь (PriorityQueue), очередь FIFO (Queue) и стек LIFO (Stack). Каждая структура хранит указатель на головной (и, при необходимости, на последний / верхний) элемент, а также текущий размер списка.

Приоритетная очередь реализована на основе односвязного списка, в который новый элемент вставляется не в конец, а в позицию, соответствующую его приоритету: список просматривается от головы до тех пор, пока не встретится элемент с приоритетом меньше вставляемого, после чего новый элемент вставляется перед ним. За счёт этого голова списка всегда содержит элемент с максимальным приоритетом, что позволяет извлекать его

за одну операцию (Pop). Помимо добавления и извлечения, для приоритетной очереди реализованы просмотр содержимого (Review), поиск элемента по имени (Find) и удаление элемента по имени (Delete).

Структура «Очередь» реализует классическую дисциплину обслуживания FIFO : элементы добавляются в конец списка (Enqueue) с помощью указателя last и извлекаются из его начала (Dequeue) с помощью указателя head. Такая реализация позволяет выполнять обе операции за время $O(1)$, поскольку не требует прохода по всему списку.

Структура «Стек» реализует дисциплину обслуживания LIFO: и добавление (Push), и извлечение (Pop) элемента выполняются только с одного конца списка — с вершины стека (top), поэтому для стека, в отличие от очереди, не требуется хранить отдельный указатель на последний элемент. Дополнительно реализована операция Peek, позволяющая просмотреть элемент на вершине стека без его извлечения.

Для удобства проверки работы всех трёх структур данных написана программа с текстовым меню, позволяющая в интерактивном режиме добавлять, извлекать, просматривать, искать и удалять элементы каждой из структур.

Схемы реализованных структур данных:

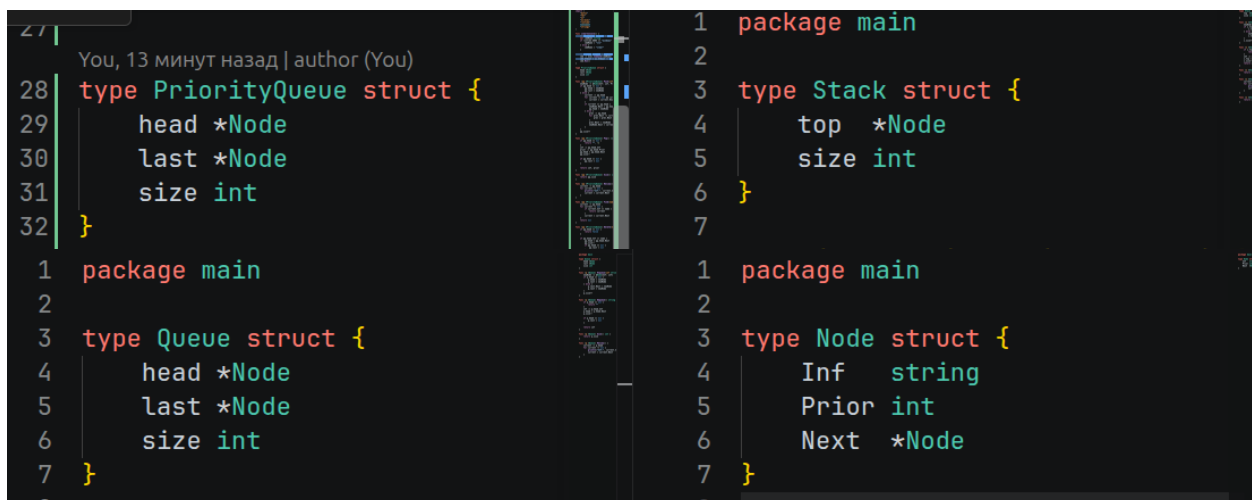


Рисунок 1 — Схема узла списка и реализованных структур данных

Результаты работы программы:

Ниже приведены результаты демонстрации работы приоритетной очереди: элементы добавлялись с разными значениями приоритета, после чего производилось их извлечение в порядке убывания приоритета.

```
[Приоритетная очередь]
1 - Добавить элемент
2 - Извлечь (макс. приоритет)
3 - Просмотр
4 - Поиск
5 - Удаление
0 - Назад
Выбор:
```

Рисунок 2 — Результаты работы приоритетной очереди

Ниже приведены результаты демонстрации работы очереди FIFO: элементы добавлялись в конец списка и извлекались из его начала в том же порядке, в котором были добавлены.

```
[Очередь]
1 - Enqueue (в конец)
2 - Dequeue (из начала)
3 - Просмотр
0 - Назад
Выбор: █
```

Рисунок 3 — Результаты работы очереди (FIFO)

Ниже приведены результаты демонстрации работы стека LIFO: элементы добавлялись на вершину стека и извлекались в порядке, обратном порядку добавления.

```
[Стек]
1 - Push (на вершину)
2 - Pop (с вершины)
3 - Peek (вершина)
4 - Просмотр
0 - Назад
Выбор: 
```

Рисунок 4 — Результаты работы стека (LIFO)

Во всех трёх случаях порядок извлечения элементов, показанный программой, совпал с ожидаемым: для приоритетной очереди — по убыванию приоритета, для очереди — в порядке добавления, для стека — в порядке, обратном порядку добавления.

Вывод:

В ходе выполнения лабораторной работы были изучены динамические структуры данных, реализуемые на основе односвязного списка. На практике были реализованы и протестированы три структуры данных: приоритетная очередь, в которой порядок элементов определяется значением приоритета; очередь, работающая по принципу FIFO; и стек, работающая по принципу LIFO. Все три структуры используют общий элемент списка (узел), что показывает, каким образом на основе одной и той же базовой единицы данных можно строить структуры с различной логикой доступа к элементам. Результаты работы программ совпали с ожидаемыми, что подтверждает корректность реализации.